

Predicting Used-Car Prices: A Regression Model

STA 244 – AF Regression Project 2026

Rupert van Vuuren

09 September 2026

1 The problem and the data

The marketplace wants to suggest a sensible asking price when a seller lists a car, and to flag adverts that look over- or under-priced. I treat this as a prediction problem and build a multiple linear regression model for **price** (in rand) from the vehicle's characteristics. Because the model will ultimately be judged on how well it predicts *unseen* cars, I use the **root mean square error (RMSE)** on a held-out set as my main yardstick, alongside the adjusted R^2 for in-sample fit.

```
cars <- read.csv("Assignment Information and Data sets-20260907/project_traindata.txt",
  stringsAsFactors = TRUE)
# listing_id is just an advert number, and model_year is a perfect mirror of vehicle_age
# (model_year + vehicle_age = 2026), so keeping both would be redundant collinearity.
cars <- cars %>% select(-listing_id, -model_year)
```

There are 2119 cars and no missing values. A quick look at the response shows the usual price pattern: most cars are cheap-to-mid range with a long tail of expensive ones, so **price** is heavily right-skewed. Its relationships with the two obvious numeric drivers, age and mileage, are also curved rather than straight — a fresh car loses value quickly and then depreciation slows down.

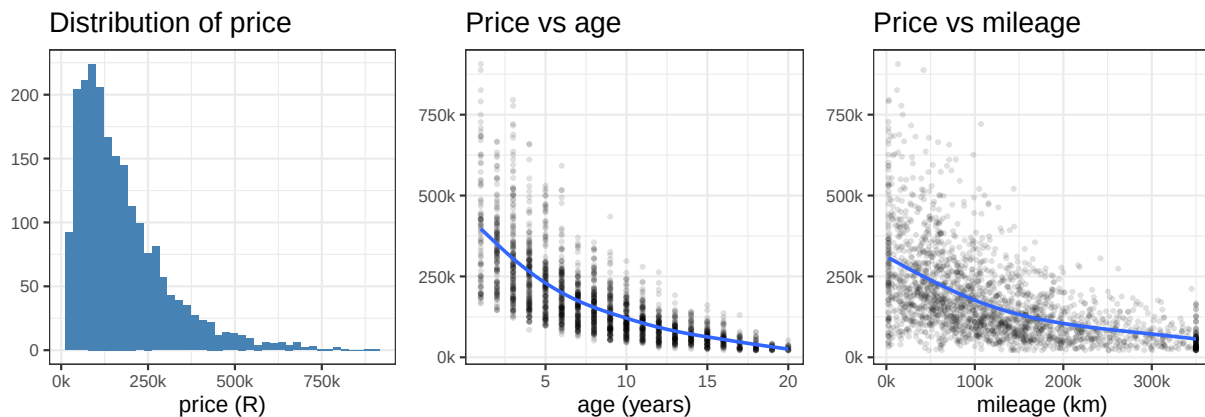


Figure 1: Price is right-skewed with a long luxury tail, and both age and mileage bend the price down at a decreasing rate.

Among the categorical variables, **segment** and **service_history** stand out as clearly price-related, and a reported accident visibly lowers the typical price. **make**, **fuel_type**, **transmission** and **seller_type** show smaller but real differences, while **colour** and **province** look almost flat against price — something I return to when selecting variables.

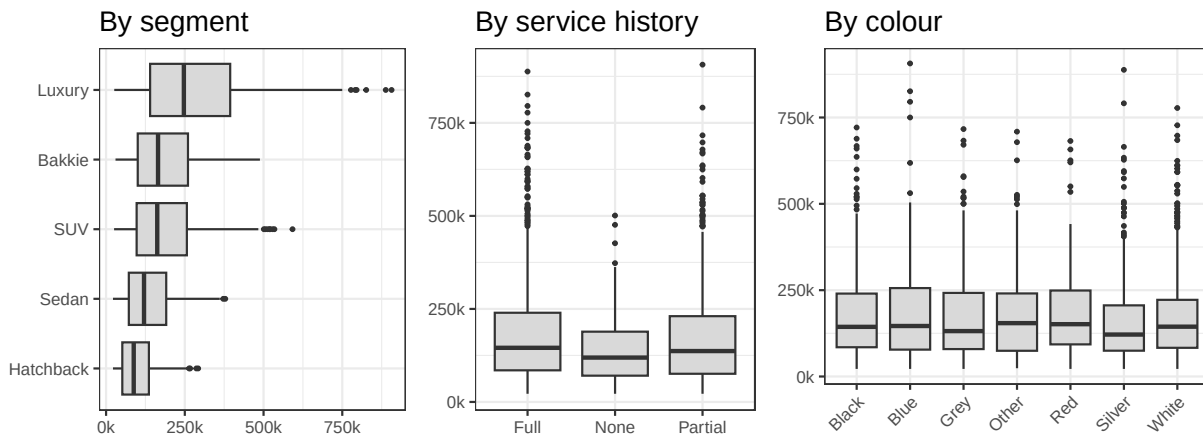


Figure 2: Price varies strongly by segment and service history and drops when an accident is reported; colour shows little relationship.

2 Training and validation split

To get an honest read on prediction accuracy I set aside part of the data before doing any modelling. I use an 80/20 split — 80% to estimate models and 20% held back purely to compare them — which keeps enough data to estimate the many factor levels reliably while leaving a validation set large enough to give a stable RMSE. I fix the seed so the split (and every result that follows) is reproducible.

```
set.seed(244)
i      <- sample(nrow(cars), floor(0.80 * nrow(cars)))
train <- cars[i, ]
valid  <- cars[-i, ]
c(train = nrow(train), validation = nrow(valid))
```

```
##      train validation
##      1695         424
```

3 Building the model

Transforming the response. The skew and the curved relationships above both point to modelling $\log(\text{price})$ rather than price . To check this properly rather than by eye, I run a Box-Cox analysis on the full model. The estimated power is $\hat{\lambda} \approx 0.06$ with a confidence interval covering 0, so the log ($\lambda = 0$) is the transformation the data support. It also has three practical pay-offs: it straightens out the age and mileage curves, makes the error spread roughly constant, and guarantees the model never predicts a negative price.

```
bc <- boxCox(lm(price ~ ., data = train), plotit = FALSE)
bc$x[which.max(bc$y)] # ~ 0.06 -> log transform
```

Choosing variables. Starting from a model with every predictor, both drop1 F -tests and stepwise selection by AIC agree that `colour` and `warranty_months` add nothing ($p \approx 0.5$) — exactly what the flat colour boxplot suggested. Everything else is significant. I drop these two, which also gives the business a concrete saving: it can stop recording exterior colour and warranty length without hurting the price estimates.

```
full <- lm(log(price) ~ ., data = train)
drop1(full, test = "F") # colour & warranty_months not significant
step(full, direction = "both", trace = 0) # AIC keeps the same reduced set
train <- train %>% select(-colour, -warranty_months)
valid <- valid %>% select(-colour, -warranty_months)
```

I did consider collapsing rare categories into an “Other” group, but every level of `make` (smallest 81 cars) and `province` is well populated and jointly significant, so grouping would only throw away information.

Adding shape and interactions. From the exploration I let age and mileage enter as quadratics to capture the flattening depreciation, and — since the brief hints that a predictor’s effect can differ by segment — I allow the age effect to vary by segment (luxury cars lose value faster than bakkies). I compare each addition on the validation set, using the log-normal correction $\exp(\hat{\sigma}^2/2)$ when back-transforming so the predictions target the mean rand price rather than the median.

```
forms <- list(
  "additive"      = log(price) ~ .,
  "+ age^2"       = log(price) ~ . - vehicle_age + poly(vehicle_age, 2),
  "+ age^2, mile^2" = log(price) ~ . - vehicle_age - mileage +
    poly(vehicle_age, 2) + poly(mileage, 2),
  "+ segment:age" = log(price) ~ . - vehicle_age - mileage +
    poly(vehicle_age, 2) + poly(mileage, 2) + segment:poly(vehicle_age, 2)
)
compare <- sapply(forms, function(f) {
  m <- lm(f, data = train); s2 <- summary(m)$sigma^2
  c(adjR2      = summary(m)$adj.r.squared,
    trainRMSE = rmse(train$price, exp(predict(m, train)) * exp(s2 / 2)),
    validRMSE = rmse(valid$price, exp(predict(m, valid)) * exp(s2 / 2)))
})
data.frame(adjR2 = round(compare["adjR2", ], 3),
  trainRMSE = round(compare["trainRMSE", ]),
  validRMSE = round(compare["validRMSE", ]))
```

##	adjR2	trainRMSE	validRMSE
## additive	0.951	34264	35679
## + age^2	0.951	34259	35844
## + age^2, mile^2	0.951	34214	35710
## + segment:age	0.953	33378	33822

Each step lowers the validation RMSE, and the segment-by-age interaction gives the best result without pulling the training and validation errors apart, so it is not just fitting noise. I tried several further interactions (segment with mileage, engine size and transmission, and a `make×segment` term); none improved the validation RMSE and the `make×segment` version clearly over-fit, so I stopped here.

```
final <- lm(forms[["+ segment:age"], ], data = train)
sigma2 <- summary(final)$sigma^2
```

4 Checking the assumptions

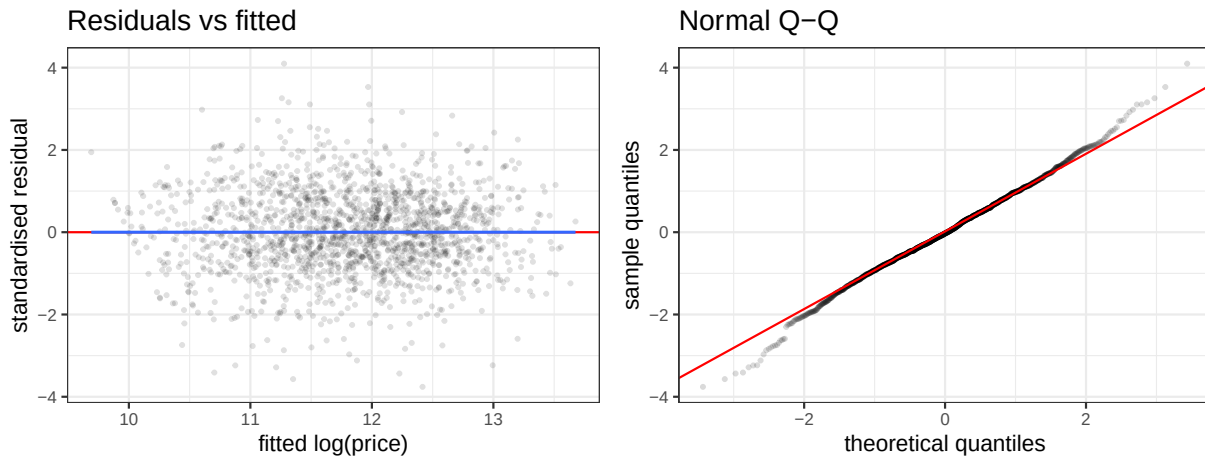


Figure 3: Residuals scatter evenly around zero with roughly constant spread, and the Q-Q plot follows the line closely apart from slightly heavy tails.

The residual plot shows no leftover curve and a fairly even band, so the linearity and constant-variance assumptions hold up well on the log scale. The Q-Q plot is close to straight with mildly heavy tails; a formal Anderson-Darling test does reject perfect normality ($p \approx 0.003$), but with 1695 observations the coefficient estimates and their tests are reliable regardless (central limit theorem), so I am comfortable with this. There is no multicollinearity problem either — every variance-inflation factor on the additive model is below about 4.5.

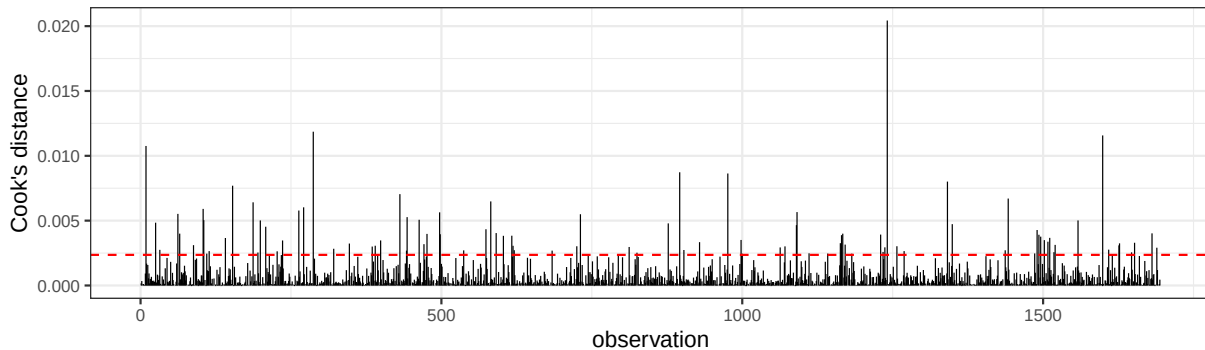


Figure 4: A handful of high-priced luxury cars are mildly influential, but none dominates the fit, so nothing is removed.

The most influential points are expensive, low-mileage luxury cars. I checked them individually; they are plausible genuine adverts, not data-entry errors, so I keep them — the model should be able to price such cars, and removing them just to flatter the fit would be dishonest.

5 What the model says, and how the business uses it

Because the response is logged, each coefficient β translates into a $100(e^\beta - 1)\%$ change in price for a one-unit change in that variable, holding the rest fixed. The main drivers are exactly the ones a buyer would expect:

Factor	Effect on price
No service history (vs full)	-16.3%
Accident reported (vs none)	-10.7%
Engine size (+1 litre)	+9.8%
Manual (vs automatic)	-4.3%
Private seller (vs dealership)	-3.8%
Each extra previous owner	-1.9%

Age and mileage dominate the price (their quadratic terms trace the steep-then-flattening depreciation curve), and **segment** shifts the whole price level — a hatchback sells for around 44% less than a comparable bakkie, while luxury cars sit highest. On top of that, condition and history matter: a car with no service record loses about 16% and a reported accident about 11%, so honest, well-serviced cars command a clear premium. Bigger engines and dealership sales lift the price, and each extra previous owner shaves a little off.

For the marketplace this gives a ready-made pricing tool: for a new advert it returns a recommended price $\hat{p} = \exp(\hat{\eta}) \exp(\hat{\sigma}^2/2)$, and any advert sitting far above or below that number can be flagged to buyers as expensive or a bargain. The model also tells the business which fields are worth collecting — age, mileage, segment, make, service and accident history carry almost all the signal, while colour and warranty length can be dropped to save data-collection cost. Its main weakness is the luxury tail: those cars carry the largest rand errors simply because small percentage misses translate into large amounts, so I would treat luxury price suggestions as more approximate.

6 Final predictions

For the competition I refit the chosen model on all 2119 cars (training plus validation) so the final predictions use every bit of data, then predict the test set.

```
final_all <- lm(forms[["+ segment:age"]], data = cars %>% select(-colour, -warranty_months))
s2_all    <- summary(final_all)$sigma^2

test <- read.csv("Assignment Information and Data sets-20260907/project_testdata.txt",
                 stringsAsFactors = TRUE)
pred <- exp(predict(final_all, test)) * exp(s2_all / 2)
write.csv(data.frame(listing_id = test$listing_id, price = round(pred)),
          "competition_predictions.csv", row.names = FALSE)
```

On the held-out validation set this model predicts price to within an RMSE of about R33,822, against a typical price near R141,500 — accurate enough to be useful while staying simple enough to explain to a seller.